INF-419: Principles of Distributed Systems

Technical University of Crete — 2025–2026

MapReduce on Kubernetes

Design Document

Team

Αστέριος Παλιγιάννης (2023030165)
Λαγγάς Ευάγγελος (2021030033)

Πασχαλίδη Ιφιγένεια (2020030153)

1. Introduction

This document describes the design of a distributed Map-Reduce platform implemented on Kubernetes, developed for INF-419 at the Technical University of Crete. The system allows authenticated users to submit parallel computation jobs defined by a user-supplied mapper and reducer, executed across a fleet of ephemeral worker containers, with results persisted to an object store.

The design follows the micro-service, 3-layer architecture required by the project: a presentation tier (UI Service + CLI), a business logic tier (Manager Service + Workers), and a data/resource tier (PostgreSQL DDS + MinIO shared file system + Keycloak authentication service), all orchestrated by Kubernetes.

The design is informed by the original MapReduce paper (Dean & Ghemawat, OSDI 2004), adapted for a Kubernetes-native deployment model where each map and reduce task runs as an independent Kubernetes Job.

2. System Concepts

Before describing the architecture, we define the principal concepts used throughout this document.

Concept	Description
User	An authenticated human who may submit Jobs and view their own results.
Administrator	A privileged user who may also create/delete users and configure the system.
Job	A parallel computation defined by a Mapper, a Reducer, an input file, and parallelism parameters (num_mappers, num_reducers).

Input Format	A data format that converts raw file bytes into key-value pairs for the mappers. We implement the JSONL (JSON Lines) format, where each line is an independent JSON object serving as one input record.
Output Format	A data format for the output produced by reducers. Output is written as JSONL files, one per reducer, stored in the MinIO output bucket.
Mapper	A user-supplied Python function that reads key-value pairs from its assigned input split and emits intermediate key-value pairs, partitioned by reducer index.
Reducer	A user-supplied Python function that reads all intermediate values for a given key (produced by all mappers) and emits final output key-value pairs.
Worker	A short-lived Kubernetes Job (container) that executes either a map or a reduce task for a specific job. Workers are created on demand and terminated on completion.
Master / Manager	A long-running replicated service that coordinates and monitors the execution of all jobs. Each job is owned by one Manager replica; on replica failure, Kubernetes restarts it and it re-attaches to its jobs.
Split	A byte-range of the input file assigned to one map task. Boundaries are snapped to newlines so no record is split across two tasks.

3. Technology Stack

This section documents all programming languages, frameworks, libraries, and infrastructure components chosen for the implementation.

3.1 Programming Language

All server-side services (UI Service, Manager Service, CLI) are implemented in Go (version 1.25). Go was chosen for its combination of high performance, excellent standard library support for HTTP and concurrency, first-class Kubernetes client libraries, and strong static typing that enables safe concurrent code. Worker containers execute user-supplied either user supplied source code (preferably go code) or docker images.

3.2 Core Frameworks & Libraries

Component	Library / Version	Purpose
HTTP Framework	`github.com/gofiber/fiber/v3` `v3.1.0`	High-performance HTTP router for UI Service and Manager Service REST APIs.
JWT Validation	`github.com/golang-jwt/jwt/v5` `v5.3.1`	Parsing and signature verification of Keycloak-issued RS256 JWTs.
UUID Generation	`github.com/google/uuid` `v1.6.0`	Generating and parsing UUID primary keys for jobs and tasks.
PostgreSQL Driver	`github.com/jackc/pgx/v5` `v5.8.0`	High-performance, type-safe PostgreSQL driver with connection pooling (pgxpool).
MinIO SDK	`github.com/minio/minio-go/v7` `v7.0.99`	S3-compatible object storage client for file uploads, ranged GETs, and presigned URLs.
DB Migrations	`github.com/pressly/goose/v3` `v3.27.0`	SQL migration management; migrations are embedded in the binary and run on startup.

SQL Code Gen	`sqlc v1.25.0`	Generates type-safe Go code from annotated SQL queries, eliminating manual row-scanning boilerplate.
Kubernetes Client	`k8s.io/client-go v0.29.0`	In-cluster API client for creating, monitoring, and deleting Kubernetes Jobs (workers).
CLI Framework	`github.com/spf13/cobra v1.10.2`	Hierarchical command-line interface with sub-commands (jobs, admin).
Config Loading	`github.com/spf13/viper v1.21.0`	Environment-variable and .env file configuration loading for all services.
Structured Logging	`go.uber.org/zap v1.27.1`	High-performance structured JSON logging with configurable level and format.
Concurrency Util	`golang.org/x/sync v0.19.0`	errgroup for concurrent goroutine error handling.

3.3 Infrastructure Components

Component	Software / Version	Role in System
Authentication Service	Keycloak 26.1.4	Full-featured Identity Provider supporting OpenID Connect and OAuth2. Manages the "mapreduce" realm with "user" and "admin" roles. Issues RS256-signed JWTs.
Distributed Data Service	PostgreSQL 16-alpine	Primary relational database storing all job and task state. Deployed with streaming replication (one primary, one hot-standby replica) for read scaling and fault tolerance.
Shared File System	MinIO (latest)	S3-compatible object store. Houses four buckets: input (user data), code (mapper/reducer scripts), jobs (intermediate shuffle files), output (final results).
Container Orchestrator	Kubernetes (minikube)	Manages all service deployments, StatefulSets, Jobs, ConfigMaps, Secrets, Services, and RBAC. Workers run as Kubernetes Jobs with TTL cleanup.
Worker Runtime	Python 3.x (container)	User-supplied mapper and reducer scripts execute inside a custom worker Docker image. The image includes Python and a MinIO client for fetching code and data.

4. System Architecture

The platform follows the three-tier micro-service architecture specified in the assignment. Figure 1 shows all services and their relationships.

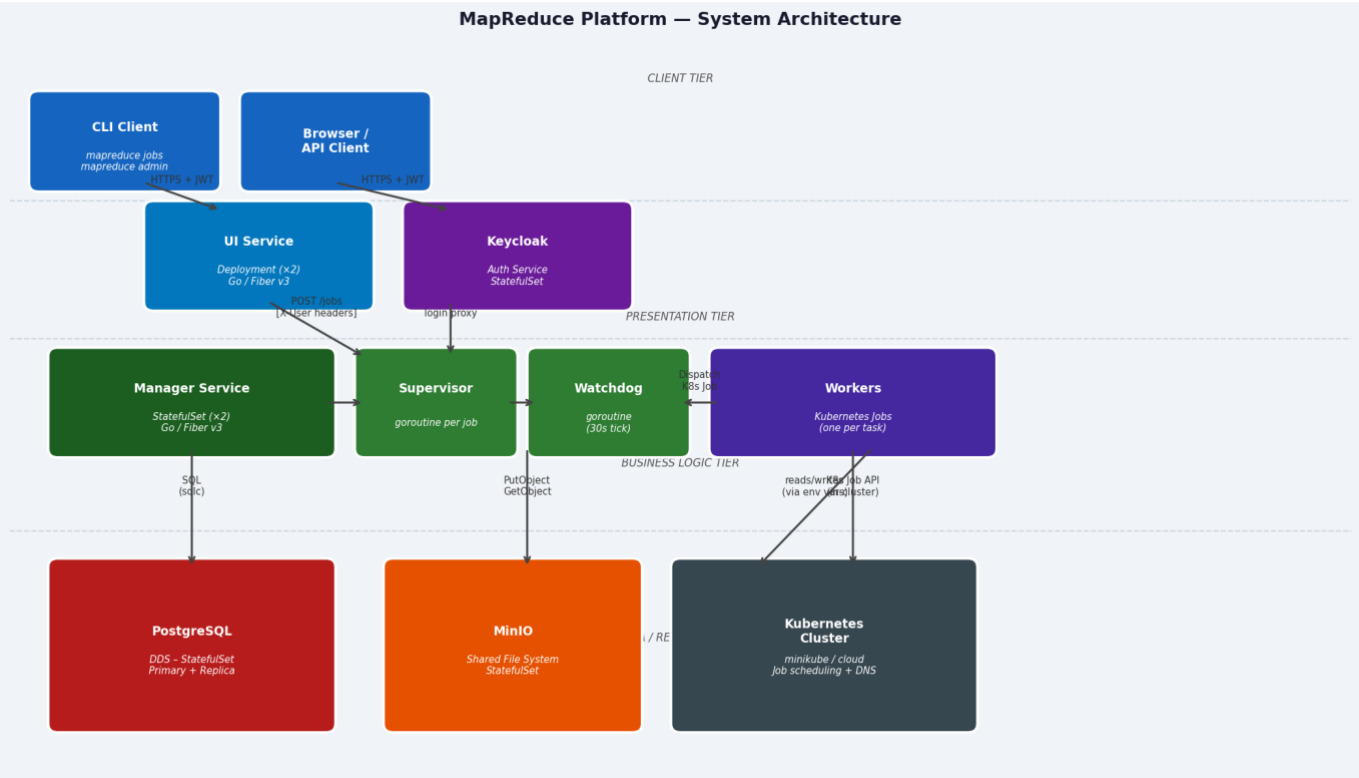

Figure 1. MapReduce Platform — System Architecture Overview

The three tiers are:

- Client Tier: The CLI client (mapreduce jobs / mapreduce admin) and any HTTP client that interacts with the UI Service via REST+JWT.
- Presentation Tier: The stateless UI Service validates JWT tokens, handles file uploads to MinIO, and proxies authenticated requests to the Manager. Keycloak provides the Identity Provider.
- Business Logic Tier: The stateful Manager Service orchestrates job execution. It splits inputs, dispatches Workers as Kubernetes Jobs, and drives the job state machine through a per-job Supervisor goroutine. A Watchdog goroutine detects stalled tasks.
- Data/Resource Tier: PostgreSQL stores all persistent state (jobs, map tasks, reduce tasks). MinIO stores all binary data (input files, code, intermediate files, outputs). The Kubernetes cluster schedules and runs Worker Jobs.

4.1 Microservices Overview

Service	K8s Kind	Responsibilities
CLI Client	—	User-facing command-line tool. Handles login, file uploads, job submission, job monitoring, and admin operations.
UI Service	Deployment (×2)	JWT validation (via Keycloak JWKS), multipart file upload to MinIO, request routing to Manager replicas, admin user management via Keycloak Admin API.
Keycloak	StatefulSet (×1)	User authentication (direct-grant flow), JWT issuance (RS256), realm/role management. Stores user data in the shared PostgreSQL instance.

Manager Service	StatefulSet (×2)	Job lifecycle orchestration, input splitting, Kubernetes Job dispatch for workers, task state management, startup recovery, stale-task watchdog.
Worker (Map)	Kubernetes Job	Downloads mapper script and input split from MinIO, executes the mapper, writes partitioned intermediate files to MinIO jobs bucket, notifies Manager on completion.
Worker (Reduce)	Kubernetes Job	Downloads reducer script and all intermediate files for its partition from MinIO, executes the reducer, writes output to MinIO output bucket, notifies Manager.
PostgreSQL (DDS)	StatefulSet ×2 (primary+replica)	Persistent storage for all job and task state. Primary handles writes; replica handles reads (scales read-heavy queries). Streaming WAL replication keeps replica in sync.
MinIO (SFS)	StatefulSet (×1)	S3-compatible object store for all binary data: user input files, mapper/reducer scripts, intermediate shuffle data, and final output files.

5. API Specification

This section documents the REST API exposed by each microservice. All APIs return JSON. Authentication is handled by the UI Service, which forwards trusted headers to internal services.

5.1 UI Service REST API

Base URL: http://ui.<minikubeIP>.nip.io (ingress, external access)

All endpoints except POST /auth/login require a valid Bearer JWT token in the Authorization header.

Method	Path	Auth	Description
POST	`/auth/login`	Public	Proxy Keycloak direct-grant flow. Body: {username, password}. Returns JWT access token.
POST	`/files/input`	user, admin	Upload input data file (multipart). Returns {path, bucket, size}. Path used as input_path in POST /jobs.
POST	`/files/code`	user, admin	Upload mapper or reducer Python script (multipart). Returns {path, bucket, size}.
GET	`/jobs`	user, admin	List jobs. Regular users see only their own jobs; admins see all.
POST	`/jobs`	user, admin	Submit a new job. Body: {mapper_path, reducer_path, input_path, num_mappers, num_reducers, input_format}. Returns job record.
GET	`/jobs/:id`	user, admin	Get job details. Users can only access their own jobs.
DELETE	`/jobs/:id`	user, admin	Cancel a running job. Translates to POST /jobs/:id/cancel on the Manager.
GET	`/jobs/:id/output`	user, admin	List MinIO output paths for a completed job.
GET	`/admin/jobs`	admin only	List all jobs across all users (admin view).
GET	`/admin/users`	admin only	List all users in the Keycloak realm.
POST	`/admin/users`	admin only	Create a new Keycloak user. Body: {username, email, password, role}.
GET	`/admin/users/:id`	admin only	Get user details from Keycloak by user ID.
DELETE	`/admin/users/:id`	admin only	Delete a user from Keycloak.
POST	`/admin/users/:id/roles`	admin only	Assign a realm role to a user. Body: {role: "user"|"admin"}.
GET	`/healthz`	Public	Health check endpoint. Returns {status: "ok"}.

5.2 Manager Service Internal API

Base URL: http://manager-api.mapreduce.svc.cluster.local:8080 (ClusterIP, internal only)

This service is not directly accessible from outside the cluster. The UI Service forwards authenticated requests with X-User-Sub, X-User-Email, and X-User-Roles headers.

Method	Path	Description
POST	`/jobs`	Create and start a new job. Launches a Supervisor goroutine. Returns 201 with job record.

GET	`/jobs`	List jobs. Filters by owner unless caller has admin role.
GET	`/jobs/:id`	Get job details. Enforces ownership (admin can access any job).
POST	`/jobs/:id/cancel`	Cancel a job (sets status to CANCELLED if not already terminal).
GET	`/jobs/:id/output`	Returns reduce task output paths for a COMPLETED job.
GET	`/admin/jobs`	Admin-only listing of all jobs.

5.3 Worker Callback API

Workers call these endpoints on the Manager ClusterIP service to report their outcome. No authentication is required (intra-cluster pod-to-pod, not reachable from outside).

Method	Path	Description
POST	`/tasks/map/:id/complete`	Map task succeeded. Body: {output_locations: [{reducer_index, path}]}. Stores locations in DB, notifies Supervisor.
POST	`/tasks/map/:id/fail`	Map task failed. Increments retry_count or permanently marks FAILED if retries exhausted. Notifies Supervisor.
POST	`/tasks/reduce/:id/complete`	Reduce task succeeded. Body: {output_path: "output/jobs/.../part-N.jsonl"}. Notifies Supervisor.
POST	`/tasks/reduce/:id/fail`	Reduce task failed. Increments retry or permanently marks FAILED. Notifies Supervisor.

6. Use Cases

This section describes the main use cases of the system. Each use case includes a UML interaction (sequence) diagram showing the services involved.

6.1 UC-1: User Authentication (Single Sign-On)

Actors: User (via CLI) | Pre-condition: user account exists in Keycloak | Post-condition: JWT token stored in ~/.mapreduce/config.json

The user runs mapreduce login --server <url> --username <name>. The CLI collects the password (masked), posts credentials to the UI Service /auth/login, which proxies a direct-grant request to Keycloak. On success, the JWT is persisted to the local config file and reused for all subsequent commands, satisfying the Single Sign-On requirement. The token contains realm_access.roles and is valid for 24 hours (configurable via Keycloak realm settings).

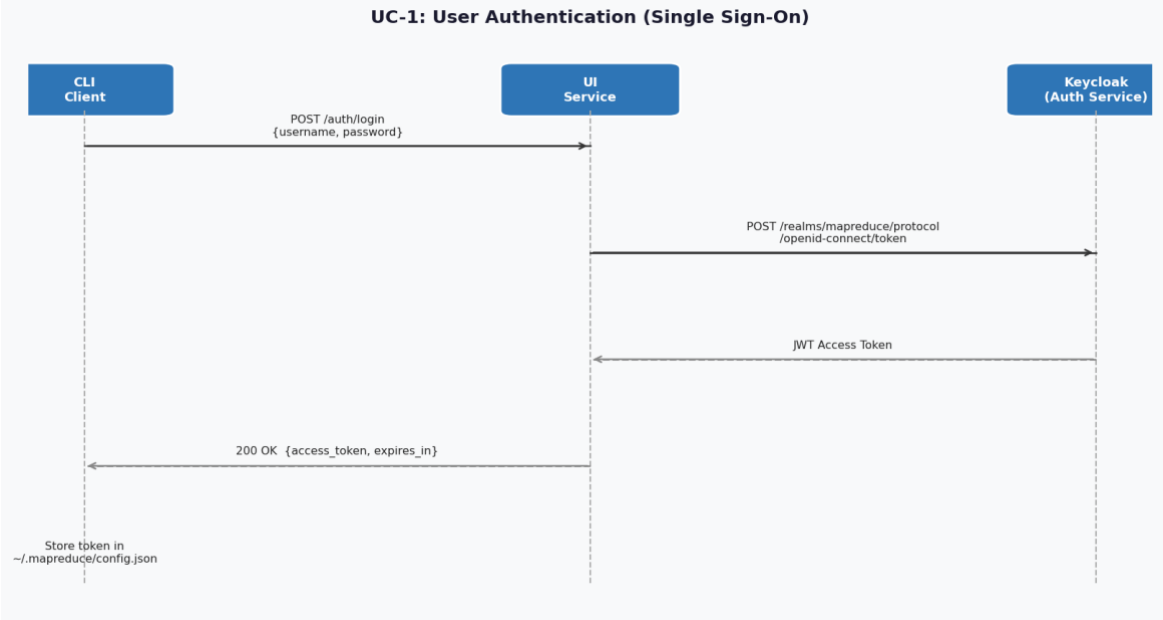

Figure 2. UC-1: User Authentication Sequence Diagram

6.2 UC-2: Job Submission

Actors: User (via CLI) | Pre-condition: user is authenticated | Post-condition: job record in PostgreSQL with status SUBMITTED, Supervisor goroutine running

Job submission is a multi-step process managed by the CLI. First, the input file and both script files are uploaded to MinIO via the UI Service. The returned MinIO object paths are then included in the POST /jobs request body. The UI Service hashes the caller's user ID (using FNV-32a) to consistently route the submission to a specific Manager StatefulSet replica (e.g., manager-0 or manager-1), ensuring ownership is stable. The Manager creates the job record, launches a Supervisor goroutine, and returns the job ID.

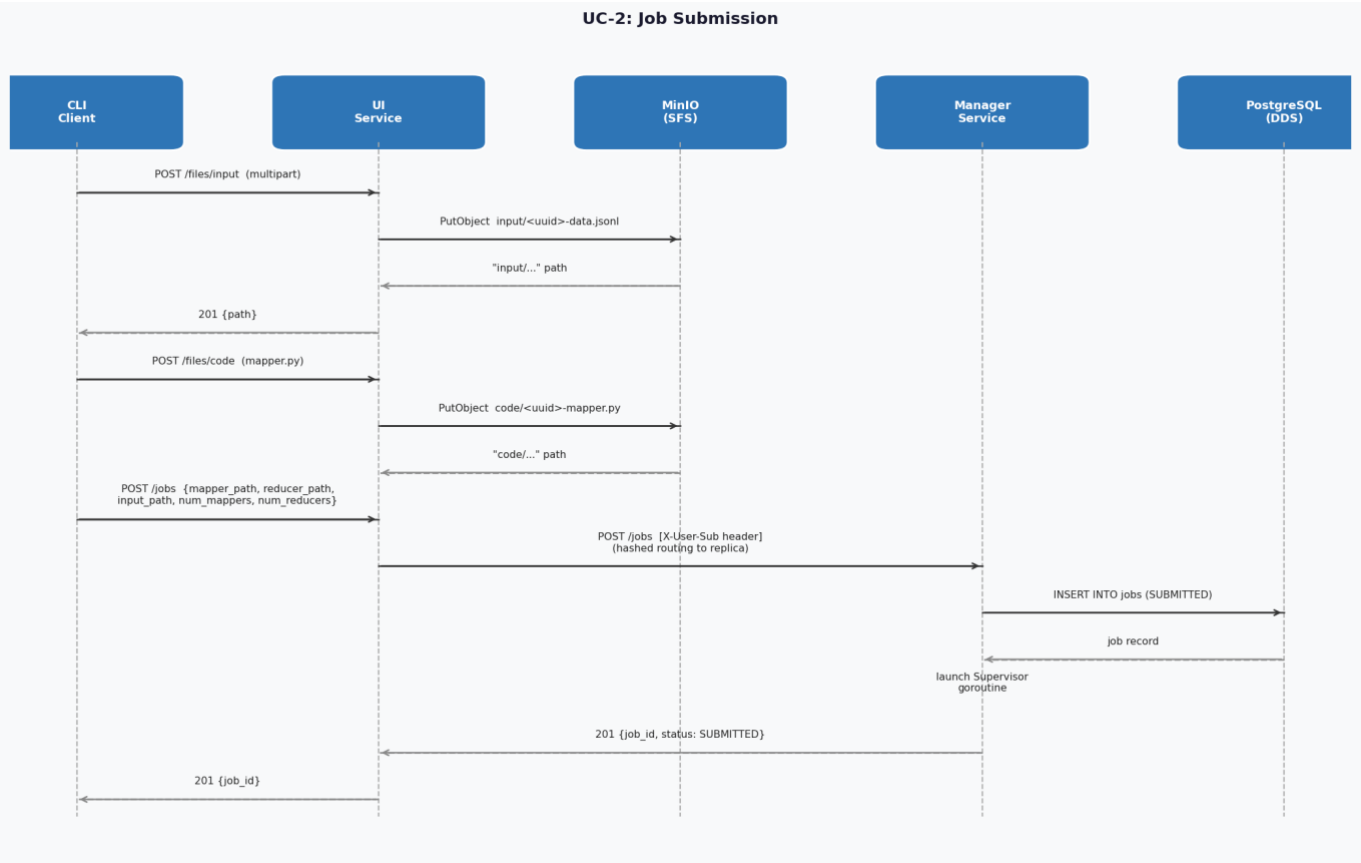

Figure 3. UC-2: Job Submission Sequence Diagram

6.3 UC-3: Map Phase Execution

Actors: Manager/Supervisor goroutine, Kubernetes, Worker pods | Pre-condition: job status = SUBMITTED | Post-condition: job status = MAP_PHASE (or REDUCE_PHASE if all maps complete)

The Supervisor goroutine drives a state machine. On detecting status SUBMITTED, it calls doSplit(), which first sets status to SPLITTING, then uses the Splitter component to compute byte-range splits from the MinIO input object. Each split becomes a map_task row (PENDING). The status then advances to MAP_PHASE and the Supervisor dispatches all pending tasks by creating Kubernetes Jobs via the k8s.io/client-go in-cluster client. When a Worker completes, it POSTs to /tasks/map/:id/complete, which marks the task COMPLETED in PostgreSQL and notifies the Supervisor via a channel. Once all map tasks are COMPLETED, the Supervisor transitions to the reduce phase.

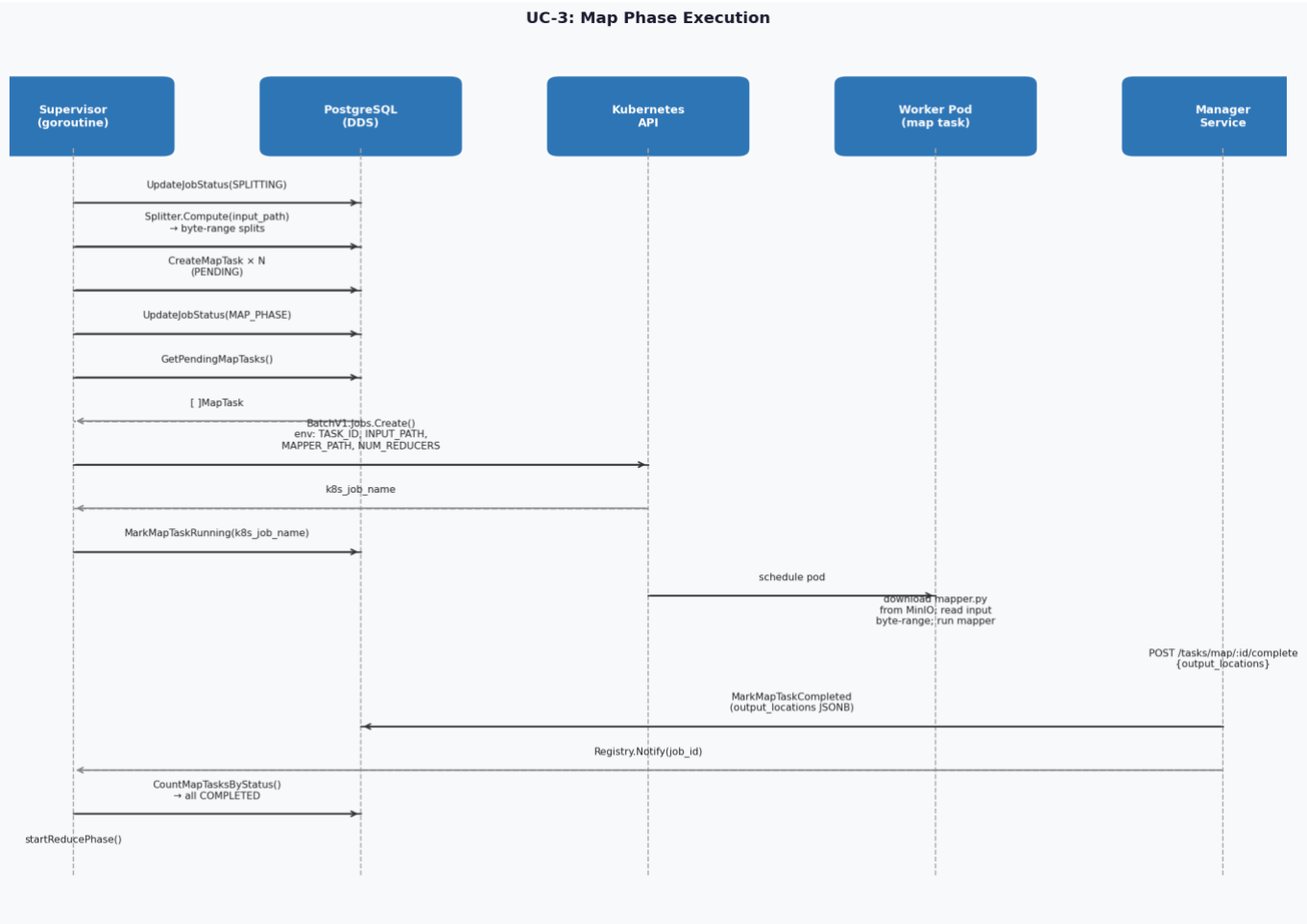

Figure 4. UC-3: Map Phase Execution Sequence Diagram

6.4 UC-4: Reduce Phase Execution

Actors: Manager/Supervisor goroutine, Kubernetes, Worker pods | Pre-condition: all map tasks COMPLETED | Post-condition: job status = COMPLETED

On entering the reduce phase, the Supervisor creates one reduce_task row per reducer index. For each task, it queries the output_locations JSONB columns of all completed map tasks, filters entries by reducer_index, and passes the resulting list as the INPUT_LOCATIONS environment variable to the Kubernetes Job. The reduce Worker downloads all its input files from MinIO, runs the user reducer, and writes the merged output as output/jobs/<job_id>/part-N.jsonl. On all reduce tasks completing, the job status is set to COMPLETED.

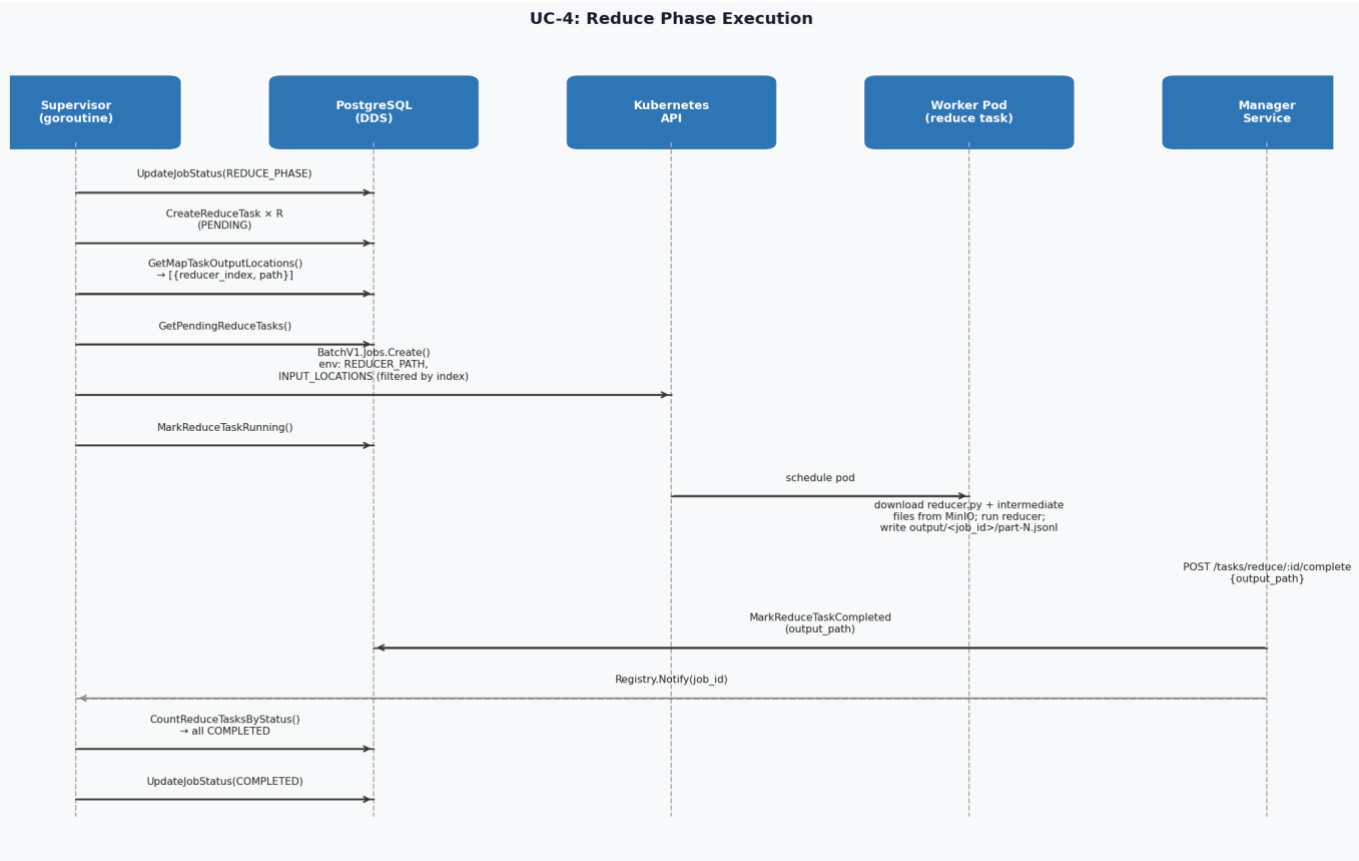

Figure 5. UC-4: Reduce Phase Execution Sequence Diagram

6.5 UC-5: Worker Failure & Recovery

Actors: Watchdog goroutine, PostgreSQL, Kubernetes, Supervisor goroutine | Pre-condition: a Worker pod has crashed or exceeded the timeout | Post-condition: task is re-queued for dispatch

Each Manager replica runs a Watchdog goroutine that scans for stale tasks every 30 seconds. A task is considered stale if it has been in RUNNING status for more than TaskTimeoutSeconds (default: 300 s). The Watchdog performs a best-effort delete of the Kubernetes Job, increments the task's retry_count, and resets its status to PENDING. It then notifies the owning job's Supervisor via the Registry channel, triggering an immediate re-dispatch. If a task's retry_count reaches the maximum (default: 3), it is permanently marked FAILED and the job eventually fails. Workers can also self-report failure by calling POST /tasks/map/:id/fail, which triggers the same retry logic immediately without waiting for the watchdog timeout.

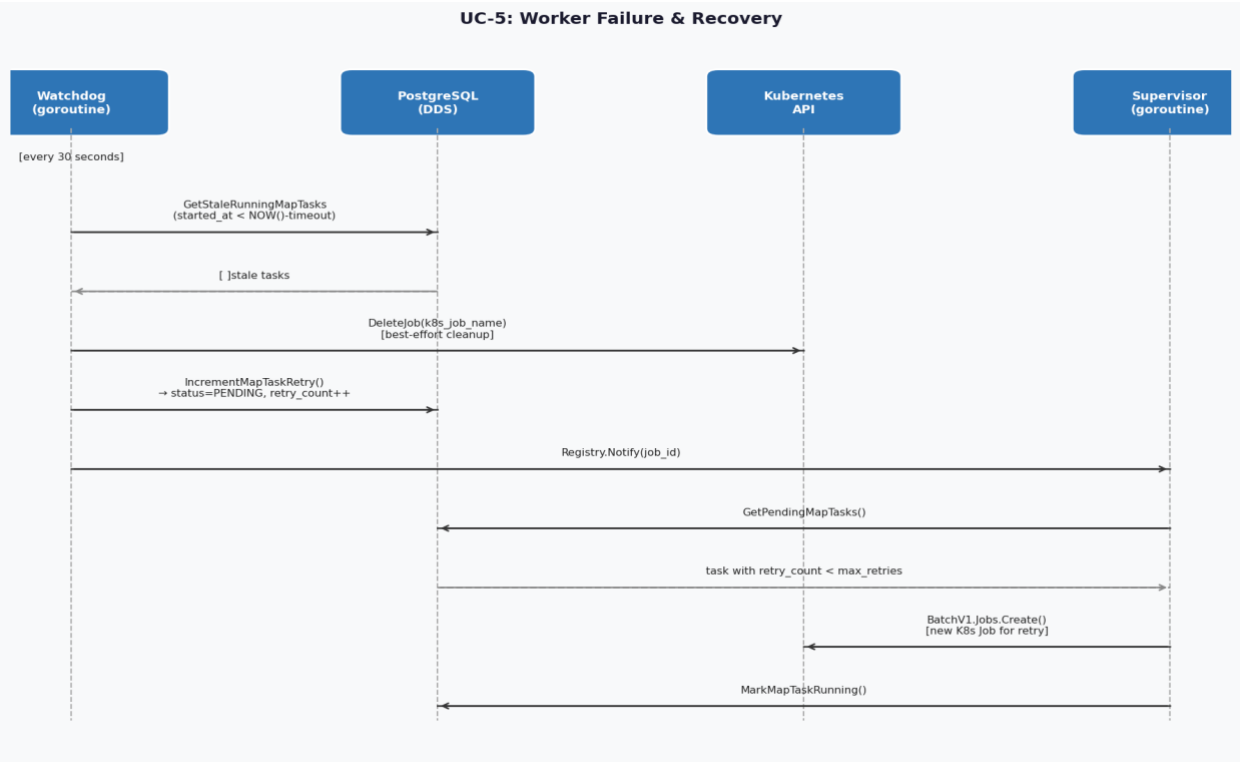

Figure 6. UC-5: Worker Failure & Recovery Sequence Diagram

6.6 UC-6: Retrieve Job Output

Actors: User (via CLI) | Pre-condition: job status = COMPLETED | Post-condition: user has MinIO object paths for all output part files

The user polls GET /jobs/:id to observe status changes, then calls GET /jobs/:id/output on a completed job. The Manager queries reduce_task.output_path for all completed reduce tasks and returns the list of MinIO paths. The user can then download the output files using the MinIO command-line client (mc) or an S3-compatible API.

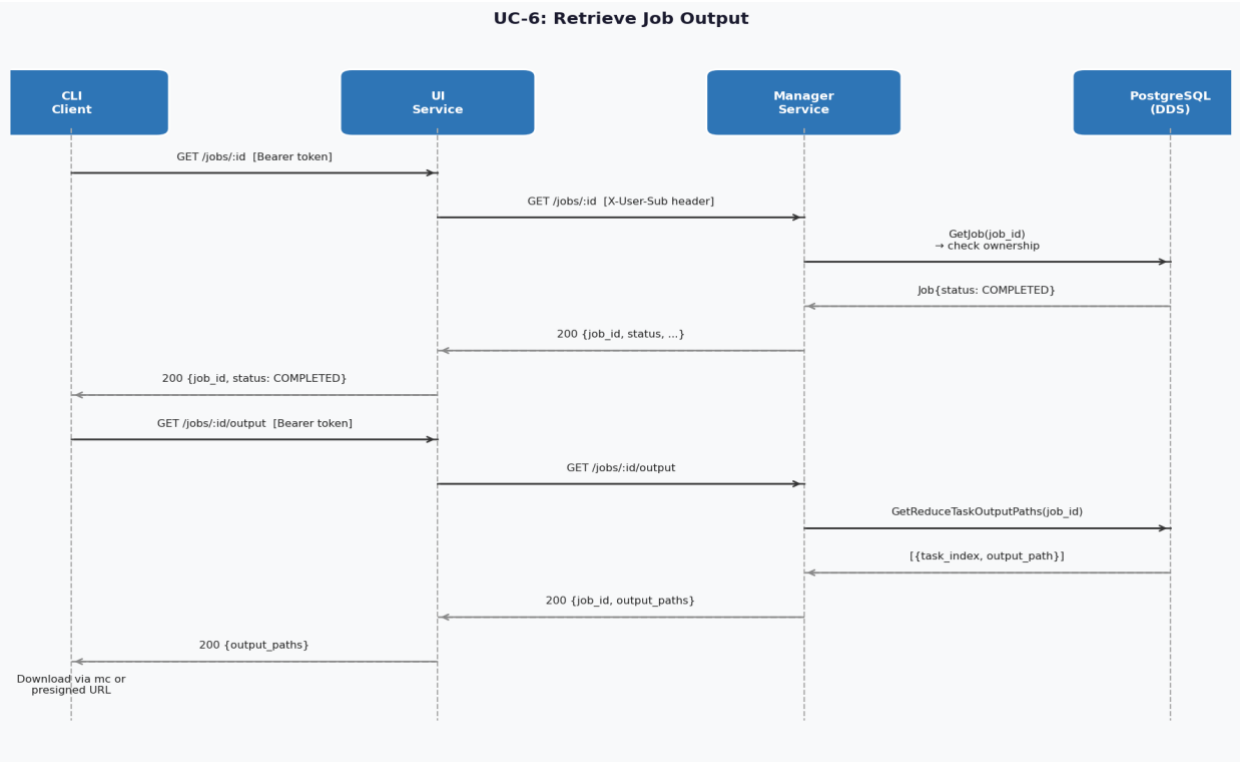

Figure 7. UC-6: Retrieve Job Output Sequence Diagram

7. Mapping Map-Reduce Jobs to Kubernetes Entities

This section specifies exactly how each concept in the Map-Reduce model corresponds to a Kubernetes resource.

7.1 Persistent Services

Service	Kubernetes Kind	Configuration Details
UI Service	Deployment	2 replicas. Rolling update strategy (maxUnavailable:1). Topology spread constraint across nodes. Liveness and readiness probes on GET /healthz.
Manager Service	StatefulSet	2 replicas. OrderedReady pod management. Stable DNS per pod: manager-N.manager-headless.mapreduce.svc.cluster.local. Stable pod name injected as MY_REPLICA_NAME via Downward API. Used as owner_replica in the jobs table.
Keycloak	StatefulSet	1 replica. Realm imported at startup via --import-realm from ConfigMap-mounted JSON. Uses PostgreSQL as backend database.
PostgreSQL Primary	StatefulSet	1 replica. PVC for data (5 Gi). Init scripts create replication role. WAL parameters passed as server args.
PostgreSQL Replica	StatefulSet	1 replica. Init container runs pg_basebackup to clone primary. Main container starts as hot-standby. Used for read-only queries.
MinIO	StatefulSet	1 replica. PVC for data (10 Gi). Post-start Job (minio-init) creates the 4 required buckets (input, code, jobs, output).

7.2 Worker Jobs (Map and Reduce Tasks)

Each individual map or reduce task is executed as a separate Kubernetes Job. This mapping ensures complete isolation between tasks and leverages Kubernetes' built-in pod scheduling, resource management, and node affinity.

Kubernetes Field	Value / Description
Kind	batch/v1 Job
Name	map-<task_id[:8]>-<index> or red-<task_id[:8]>-<index> (max 63 chars)
Namespace	mapreduce
completions	1 (each Job runs exactly one pod to completion)
parallelism	1
backoffLimit	0 — retries are managed at the application level by the Supervisor/Watchdog, not by Kubernetes
ttlSecondsAfterFinished	300 — completed pods are auto-cleaned after 5 minutes
restartPolicy	Never — prevents Kubernetes from masking application-level failures
container image	mirstar13/mapreduce-worker:latest (custom Docker image with Python, MinIO client)
imagePullPolicy	Always

7.2.1 Map Worker Environment Variables

Variable	Value / Source
TASK_ID	UUID of the map_task row
TASK_TYPE	"map"
JOB_ID	UUID of the parent job
TASK_INDEX	Integer index of this split (0-based)
INPUT_PATH	JSON: {file, offset, length} — byte-range of the input file to process
MAPPER_PATH	MinIO key of the mapper script in the code bucket
NUM_REDUCERS	Number of reducer partitions; worker uses hash(key) % NUM_REDUCERS for partitioning
MANAGER_URL	http://manager-api.mapreduce.svc.cluster.local:8080 — callback endpoint
MINIO_ENDPOINT	minio-api.mapreduce.svc.cluster.local:9000
MINIO_ACCESS_KEY MINIO_SECRET_KEY	Injected from the mapreduce-secrets Kubernetes Secret

7.2.2 Reduce Worker Environment Variables

Variable	Value / Source
TASK_ID	UUID of the reduce_task row
TASK_TYPE	"reduce"
TASK_INDEX	Reducer partition index
REDUCER_PATH	MinIO key of the reducer script in the code bucket
INPUT_LOCATIONS	JSON array of {reducer_index, path} objects — all intermediate files for this partition from all map tasks
MANAGER_URL	http://manager-api.mapreduce.svc.cluster.local:8080
MINIO_BUCKET_OUTPUT	"output" — destination bucket for part-N.jsonl files

7.3 Kubernetes Services

Service Name	Type	Port	Purpose
ui-api	ClusterIP	8081	Internal access from other pods
ui-ingress	Ingress	8081	External / CLI access
manager-api	ClusterIP	8080	Load-balanced endpoint for UI → Manager and Worker callbacks
manager-headless	Headless	8080	Stable per-pod DNS for UI → specific replica routing
keycloak	ClusterIP	8080	Internal auth service access
keycloak-ingress	Ingress	8080	Dev / admin console access
postgres-primary	Headless	5432	Write DSN for Manager and Keycloak

postgres-replica	Headless	5432	Read-only DSN for read-heavy queries
minio-api	ClusterIP	9000	Object storage API for all services
minio-ingress	Ingress	9000/9001	Dev / console access

8. Database Schema

All persistent system state is stored in a PostgreSQL database. The schema is managed by Goose migrations (embedded in the binary) and Go type-safe queries are generated by sqlc. The schema consists of three tables.

8.1 jobs

Stores the top-level record for each Map-Reduce job. One row per submitted job.

Column	Type	Nullable	Description
job_id	`UUID`	PK	Primary key, auto-generated with gen_random_uuid()
owner_user_id	`VARCHAR(255)`	NOT NULL	Keycloak subject (sub claim) of the submitting user
owner_replica	`VARCHAR(63)`	NOT NULL	Pod name of the Manager replica that owns this job (e.g. "manager-0"). Used for startup recovery.
status	`VARCHAR(32)`	NOT NULL	State machine: SUBMITTED → SPLITTING → MAP_PHASE → REDUCE_PHASE → COMPLETED | FAILED | CANCELLED
mapper_path	`VARCHAR(512)`	NOT NULL	MinIO key of mapper script in the code bucket
reducer_path	`VARCHAR(512)`	NOT NULL	MinIO key of reducer script in the code bucket
input_path	`VARCHAR(512)`	NOT NULL	MinIO key of input file in the input bucket
output_path	`VARCHAR(512)`	NOT NULL	Base MinIO path prefix for output files: output/jobs/<job_id>
num_mappers	`INTEGER`	NOT NULL	Requested number of map tasks (splits)
num_reducers	`INTEGER`	NOT NULL	Number of reduce tasks and output partitions
input_format	`VARCHAR(32)`	NOT NULL	Data format: "jsonl" (default) or "text"
submitted_at	`TIMESTAMPTZ`	NOT NULL	Timestamp of job creation
started_at	`TIMESTAMPTZ`	NULL	Set when status transitions to MAP_PHASE
completed_at	`TIMESTAMPTZ`	NULL	Set when status reaches a terminal state
error_message	`TEXT`	NULL	Human-readable failure reason (set on FAILED)

Indexes: idx_jobs_owner_user (owner_user_id), idx_jobs_owner_replica_status (owner_replica, status), idx_jobs_status (status)

8.2 map_tasks

One row per map task (one per input split). Tasks are created by the Supervisor when the job transitions to MAP_PHASE.

Column	Type	Nullable	Description
task_id	`UUID`	PK	Auto-generated primary key
job_id	`UUID`	FK, NOT NULL	References jobs(job_id) ON DELETE CASCADE
task_index	`INTEGER`	NOT NULL	0-based index of this split within the job

status	VARCHAR(32)	NOT NULL	PENDING → RUNNING → COMPLETED | FAILED
input_file	VARCHAR(512)	NOT NULL	MinIO object key of the input file
input_offset	BIGINT	NOT NULL	Byte offset of this split within the input file
input_length	BIGINT	NOT NULL	Byte length of this split
k8s_job_name	VARCHAR(63)	NULL	Kubernetes Job name assigned at dispatch time
worker_pod	VARCHAR(63)	NULL	Name of the pod that executed this task
retry_count	INTEGER	NOT NULL	Number of times this task has been retried (default 0)
started_at	TIMESTAMPTZ	NULL	Set when Kubernetes Job is created; reset on retry
completed_at	TIMESTAMPTZ	NULL	Set when task reaches COMPLETED or FAILED
output_locations	JSONB	NULL	Array of {reducer_index: int, path: string} objects. Set by the map worker on completion. Used to build reduce task inputs.

Indexes: idx_map_tasks_job_status (job_id, status), idx_map_tasks_status (status)

Unique constraint: `uq_map_task_job_index (job_id, task_index)`

8.3 reduce_tasks

One row per reduce task. Created by the Supervisor when transitioning to REDUCE_PHASE.

Column	Type	Nullable	Description
task_id	UUID	PK	Auto-generated primary key
job_id	UUID	FK, NOT NULL	References jobs(job_id) ON DELETE CASCADE
task_index	INTEGER	NOT NULL	0-based reducer partition index
status	VARCHAR(32)	NOT NULL	PENDING → RUNNING → COMPLETED | FAILED
k8s_job_name	VARCHAR(63)	NULL	Kubernetes Job name assigned at dispatch time
worker_pod	VARCHAR(63)	NULL	Name of the pod that executed this task
retry_count	INTEGER	NOT NULL	Number of retries (default 0)
started_at	TIMESTAMPTZ	NULL	Set at dispatch; reset on retry
completed_at	TIMESTAMPTZ	NULL	Set at terminal state
output_path	VARCHAR(512)	NULL	MinIO key of the output file: output/jobs/<job_id>/part-<N>.jsonl. Set on COMPLETED.

Indexes: idx_reduce_tasks_job_status (job_id, status), idx_reduce_tasks_status (status)

9. Job and Task Lifecycle

Figure 8 illustrates the state machines for both the job-level and task-level lifecycle. These state machines are the core of the Supervisor goroutine logic.

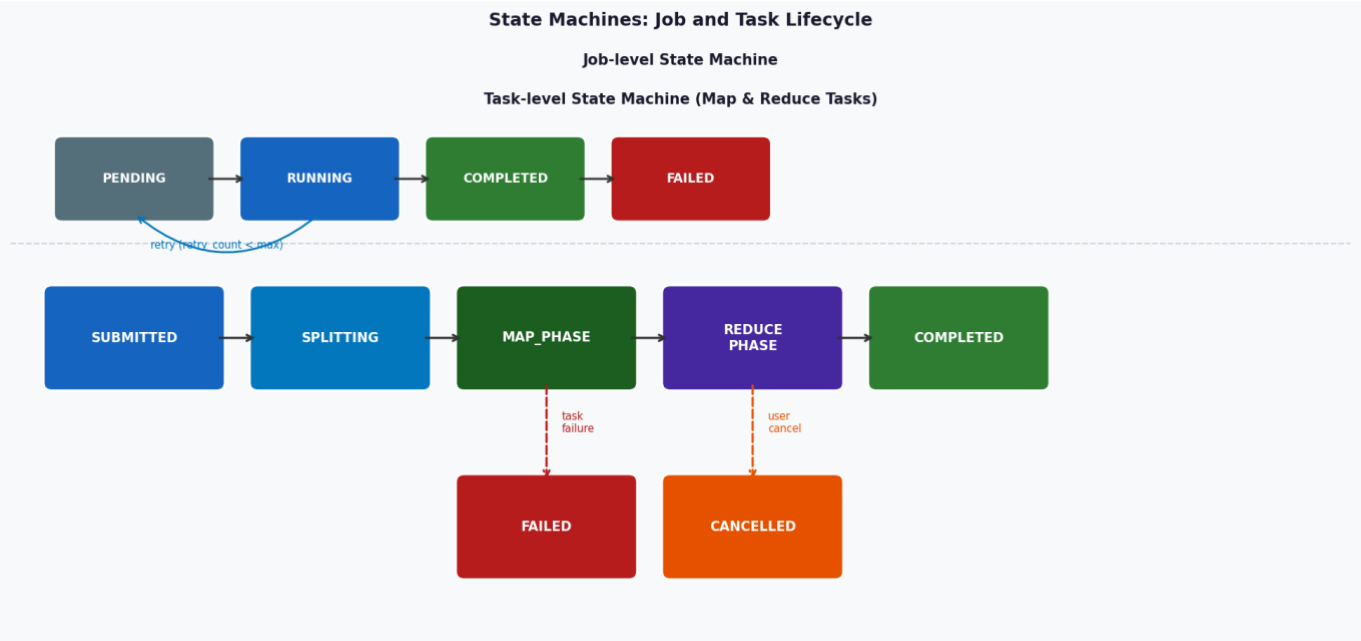

Figure 8. Job-level and Task-level State Machine Diagrams

9.1 Job-Level State Machine

Transition	Trigger
SUBMITTED → SPLITTING	Supervisor goroutine starts; UpdateJobStatus(SPLITTING) called before Splitter.Compute().
SPLITTING → MAP_PHASE	All map_task rows created; UpdateJobStatus(MAP_PHASE) called.
MAP_PHASE → REDUCE_PHASE	CountMapTasksByStatus() returns completed == total (and failed == 0).
REDUCE_PHASE → COMPLETED	CountReduceTasksByStatus() returns completed == total.
Any → FAILED	Map or reduce tasks fail permanently (retry_count >= max). FailJob() called with error message.
Any → CANCELLED	User calls DELETE /jobs/:id before job reaches terminal state.

9.2 Task-Level State Machine

Transition	Trigger
PENDING → RUNNING	Manager dispatches Kubernetes Job; MarkMapTaskRunning() / MarkReduceTaskRunning() called.
RUNNING → COMPLETED	Worker calls POST /tasks/{map|reduce}/:id/complete.
RUNNING → FAILED	Worker calls POST /tasks/{map|reduce}/:id/fail AND retry_count >= TaskMaxRetries.

RUNNING → PENDING	Watchdog detects stale task (timeout exceeded); IncrementTaskRetry() resets to PENDING.
RUNNING → FAILED	Worker self-reports failure AND retry_count >= TaskMaxRetries; OR Watchdog detects stale task AND retry_count >= max.

10. Security Design

The security design follows the principle of defence in depth, with authentication happening only at the external boundary and authorization enforced at both the UI and Manager layers.

10.1 Authentication

- All user-facing endpoints (except POST /auth/login) require a Bearer JWT token in the Authorization header.
- JWTs are issued by Keycloak using the RS256 algorithm. The UI Service validates tokens locally using a cached JWKS (refreshed every 5 minutes) from Keycloak's JWKS endpoint, avoiding a round-trip to Keycloak on every request.
- Validation checks: signature (via RSA public key matching kid header), expiry (exp), issuer (iss must equal our realm URL), and authorized party (azp must match mapreduce-ui client ID).
- Keycloak system roles (offline_access, uma_authorization, default-roles-mapreduce) are filtered out before the Identity struct is constructed. Only application roles (user, admin) are forwarded.

10.2 Authorization

- Role-based access control (RBAC) is enforced by middleware. Regular users (role: user) can only access their own jobs. Admins (role: admin) can access all jobs and user management endpoints.
- The UI Service forwards validated identity as X-User-Sub, X-User-Email, and X-User-Roles headers to the internal Manager Service. The Manager trusts these headers (internal-only, ClusterIP service, not externally reachable).
- Worker callback endpoints on the Manager (/tasks/*/complete|fail) have no authentication — they are accessible only within the cluster network. This is acceptable because workers cannot be reached from outside and run only within Kubernetes.

10.3 Secrets Management

- All credentials (database passwords, MinIO access keys, Keycloak admin passwords) are stored in a Kubernetes Secret (mapreduce-secrets) and injected into pods as environment variables.
- No credentials appear in ConfigMaps or container images.

11. Fault Tolerance

11.1 Worker Failures

Workers are stateless Kubernetes Jobs. A worker failure is handled at two levels:

- Self-reported: The worker calls POST /tasks/:id/fail, which triggers an immediate retry via IncrementTaskRetry() and a Supervisor notification.
- Timeout-detected: The Watchdog goroutine (running in each Manager replica) scans for tasks in RUNNING state whose started_at is older than TaskTimeoutSeconds (default 300 s). Stale tasks are reset to PENDING (retry_count incremented), and their Kubernetes Job is deleted.
- Maximum retries: If retry_count reaches TaskMaxRetries (default 3), the task is permanently marked FAILED. If any tasks permanently fail, the job eventually transitions to FAILED.

11.2 Manager Replica Failures

The Manager is deployed as a StatefulSet with 2 replicas. Each replica has a stable pod name (manager-0, manager-1) stored as owner_replica in the jobs table. On restart, each replica calls GetActiveJobsByReplica(myReplicaName) to find all in-flight jobs it owned and re-launches a Supervisor goroutine for each. This ensures zero job loss on pod restart.

11.3 Database Fault Tolerance

PostgreSQL is deployed with streaming replication (one primary, one hot-standby replica). Application writes go to the primary (via postgres-primary headless service) and reads can be directed to the replica (POSTGRES_DSN_READONLY). If the primary fails, manual promotion of the replica is possible (kubectl exec pg_ctl promote). All schema migrations are idempotent and managed by Goose.

11.4 UI Service Fault Tolerance

The UI Service is stateless and deployed as a Deployment with 2 replicas behind a Kubernetes Service (load-balanced). Any replica can handle any request. If one replica dies, Kubernetes automatically routes traffic to the surviving replica and restarts the failed pod.

12. Scalability

The design is architected to scale along all relevant dimensions:

- Input size: The Splitter divides any arbitrarily large input file into byte-range splits, each processed independently. The total work scales linearly with input size.
- Map parallelism: num_mappers controls how many map tasks (and Kubernetes Jobs) are created. The system can in principle handle thousands of mappers limited only by Kubernetes cluster capacity.
- Reduce parallelism: num_reducers controls output partitioning. Each reducer runs as an independent Kubernetes Job.
- Intermediate data: Intermediate shuffle files are stored in MinIO (object storage), which scales independently of the compute layer. Workers communicate only via MinIO and the Manager callback API, not directly with each other.
- Manager capacity: Multiple Manager replicas can serve different jobs simultaneously. The hashing scheme in the UI Service ensures each job is consistently routed to one replica, avoiding cross-replica coordination.

13. CLI Design

The command-line interface is built with the Cobra framework and provides two top-level command groups, as required by the assignment.

13.1 jobs Commands

Command	Description
`mapreduce login --server <url> --username <u>`	Authenticate and store JWT to ~/.mapreduce/config.json
`mapreduce jobs list`	List the calling user's jobs (table or --json output)
`mapreduce jobs get <job-id>`	Show detailed information about a specific job
`mapreduce jobs submit --input <file> --mapper <py> --reducer <py> --mappers N --reducers M`	Upload files to MinIO and submit a new job in one command
`mapreduce jobs cancel <job-id>`	Cancel a running job
`mapreduce jobs output <job-id>`	List MinIO output file paths for a completed job

13.2 admin Commands

Command	Description
`mapreduce admin jobs list`	List all jobs across all users (admin only)
`mapreduce admin users list`	List all users in the Keycloak realm
`mapreduce admin users create --username <u> --email <e> --password <p> [--role admin]`	Create a new user in Keycloak and assign a realm role
`mapreduce admin users delete <keycloak-id>`	Delete a user from Keycloak

`mapreduce admin users role <id> --role <r>`	Assign (or change) the realm role of a user

14. Deployment

The entire system is deployed to a Kubernetes namespace "mapreduce" using a set of YAML manifests. Manifests are applied in order using kubectl apply -f:

1. 00_namespace.yml — creates the mapreduce namespace
2. 01_secrets.yml — database, MinIO, and Keycloak credentials
3. 02_configmap.yml — non-secret configuration (endpoints, bucket names, etc.)
4. 03_postgres.yml — PostgreSQL primary StatefulSet, headless + NodePort service, init scripts ConfigMap
5. 04_postgres_replica.yml — PostgreSQL hot-standby StatefulSet (deploy after primary is ready)
6. 04_keycloak.yml — Keycloak StatefulSet, services, realm ConfigMap
7. 05_minio.yml — MinIO StatefulSet, services, bucket-init Job
8. 06_manager.yml — Manager StatefulSet, RBAC (ServiceAccount, Role, RoleBinding), services
9. 07_ui.yml — UI Service Deployment, services
10. 08_ingress.yml — External service access

A separate Kubernetes Job (migrate) runs Goose migrations before the Manager starts, ensuring the database schema is always up to date.

15. End-to-End Data Flow Summary

The following summarizes the complete lifecycle of a Map-Reduce job from submission to output retrieval:

11. User uploads input data file and scripts (mapper.py, reducer.py) to MinIO via the UI Service.
12. User submits a job via POST /jobs. The UI Service routes to a specific Manager replica based on a hash of the user ID.
13. Manager creates a job record in PostgreSQL (status: SUBMITTED) and starts a Supervisor goroutine.
14. Supervisor computes byte-range input splits using the MinIO StatObject + ranged GetObject API. Creates N map_task rows in PostgreSQL.
15. Supervisor dispatches N Kubernetes Jobs (map workers). Each job receives its byte-range via INPUT_PATH env var.
16. Each map worker: (a) downloads mapper.py from MinIO, (b) reads its byte-range via ranged GET, (c) runs the mapper, (d) writes M intermediate files (one per reducer) to the jobs bucket, (e) calls POST /tasks/map/:id/complete.
17. Once all N map tasks complete, Supervisor creates R reduce_task rows and dispatches R Kubernetes Jobs (reduce workers).

18. Each reduce worker: (a) downloads reducer.py from MinIO, (b) downloads all intermediate files for its partition from MinIO, (c) runs the reducer, (d) writes output/jobs/<job_id>/part-<R>.jsonl to MinIO, (e) calls POST /tasks/reduce/:id/complete.
19. Once all R reduce tasks complete, Supervisor sets job status to COMPLETED.
20. User calls GET /jobs/:id/output to retrieve the list of output file paths and downloads results via mc or presigned URLs.

Appendix: MinIO Bucket Structure

Bucket	Key Pattern	Content
input	input/<uuid>-<filename>	User-uploaded input data files
code	code/<uuid>-<filename>	User-uploaded mapper and reducer Python scripts
jobs	intermediate/jobs/<job_id>/ map-<M>-reduce-<R>.jsonl	Intermediate shuffle files written by map workers, partitioned by reducer index
output	output/jobs/<job_id>/ part-<R>.jsonl	Final output files written by reduce workers